

[9 marks]

Q11 List different types of operating system? describe any two of them.

→ Different types of operating systems are:-

① Single user

① Batch operating system ② multiprogramming

③ Time-sharing / multitasking OS

④ multiprocessing OS

⑤ Distributed OS

⑥ Network OS

⑦ Real time OS

⑧ mobile OS

Explanation:-

① Batch operating system:-

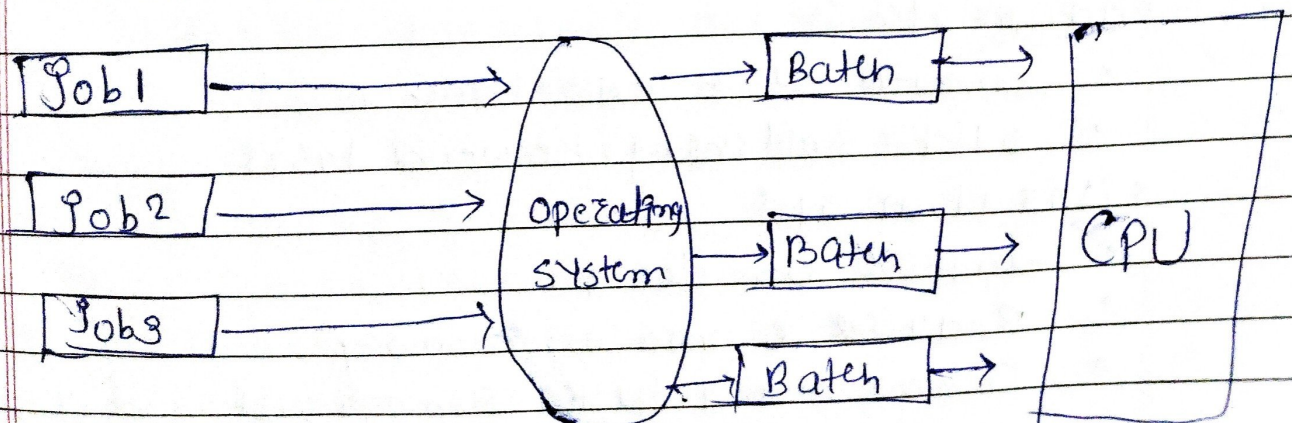
+ A Batch operating system groups similar jobs together & executes them in batches, one after another.

- It does not interact with computer directly

- User prepare job → jobs are collected →

OS processes them batch-wise

- there is no direct interaction between the user & the computer during execution.



* Advantages :-

- Increases CPU utilization
- Reduces idle time of CPU

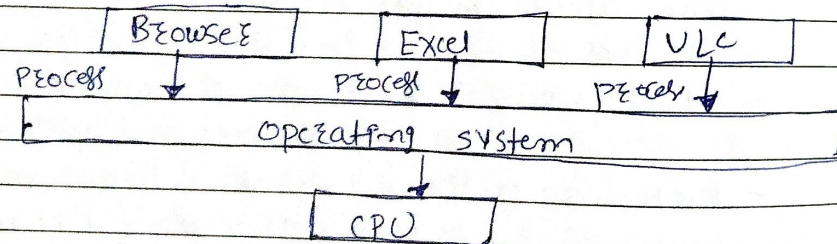
* Disadvantages :-

- No real time interaction
- Debugging is difficult

eg Exam result generation.

② multitasking OS -

- It is also called as time sharing operating system.
- It allows a single user to run multiple tasks (programs) at the same time.
- CPU every process runs in round robin manner.
- each task is given some time quantum to execute so that all works the tasks look smoothly.



* Advantages :-

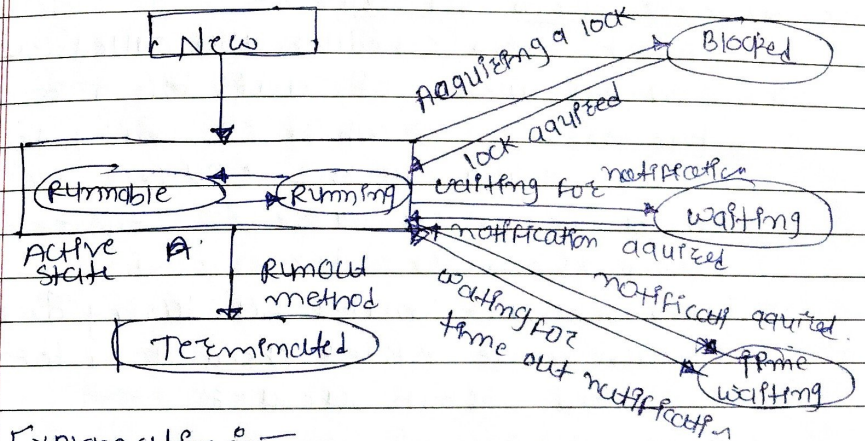
- smooth user experience
- quick switching between tasks

* Disadvantages

- Complex OS design
- Requires more memory
- If too many tasks run → system may slow down

Q2] What is meant by thread? explain thread lifecycle with diagram.

- ⇒
- A thread is a smallest unit of execution inside a process.
 - a single process can have multiple threads.
 - threads share:
 - same memory, same files, same code
 - But each thread has its own stack & PC.



Explanation :-

- ① New state :- when a thread is created it is in new state.
- ② Runnable state :- a thread is ready to run. It is moved to a Runnable state. Here thread actually be running or might be ready to run.
- ③ Blocked :- If in this state a thread is waiting for a lock or resource that is acquired by another process.
- ④ Waiting state :- when thread calls wait() or sleep() method, it will move to waiting state.

⑤ Time Waiting :- a thread is in this state until timeout is completed or until notification is received.

⑥ Terminated state :- this state defines that thread has completed its execution.

Q3] Explain following types of Schedulers.

⇒ a) Short Term Scheduler :-

- Short term scheduler also called as a CPU scheduler. It selects the process for execution & which is ready to execute & allocate the CPU for execution.
- Short term scheduler is faster than long term scheduler.
- This scheduler makes scheduling decision much more easier than long term & mid term scheduler.

b) Medium Term Scheduler :-

- It is the part of a swapping function. Sometimes it removes a process from a memory.
- If process makes an I/O request & it is in memory then operating system takes this process & puts it into suspended state. Once process becomes suspended it cannot make any process towards computation.
- In this process situation, process is removed from memory & makes free space for other process.

③ Long Term Scheduler :-

- Long term scheduler is also called as a Job scheduler.
- It determines which process are admitted to system for execution processing.
- Processes are selected from queue & loaded into main memory for execution.
- Long term scheduling is used in a real time operating systems.
- & it is ^{not} ~~never~~ ~~been~~ used in a time sharing or multitasking operating system.

Q4] Differentiate between Preemptive & non-preemptive scheduling.

→ Preemptive scheduling
① It allows a process to be interrupted in the midst of its execution, taking the CPU away & allocating it to another process.

② Process cannot be interrupted in between.

③ If high priority process arrives then it can get CPU easily. Immediately & low priority process has to wait.

non-preemptive
① CPU cannot be taken away; process keeps CPU until it finishes.

② Process can not be interrupted until it terminates itself or it time is up.

③ If high priority process arrives it must wait.

④ PT has overheads of scheduling processes. ④ It does not have overhead.

③ decisions made based on priority & time slice allocation. ⑤ deletion made by the process itself & OS follows process's instruction.

⑥ PT has faster response time. ⑥ It has slower response time.

⑦ more chances of starvation. ⑦ less chances of starvation.

⑧ ex Round Robin, Preemptive Priority, SRTF. ⑧ ex FCFS, Non-preemptive priority, SJF.

Q57 Explain multithreaded model of process control block in detail.

⇒ multithreading is a technique where a process is divided into smaller execution units called threads that runs concurrently.
- multithreaded model means a program can have multiple threads running inside it at same time.

* multithreading modes
- operating system uses user level & kernel level threads.

① many to one model :-

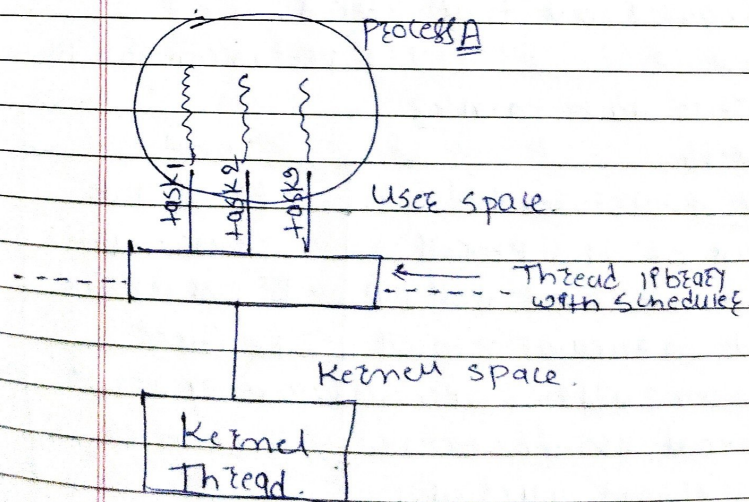
- many user threads mapped to one kernel thread.
- If one thread blocks, the entire process blocks.
- It is a simple mode but not good for performance.

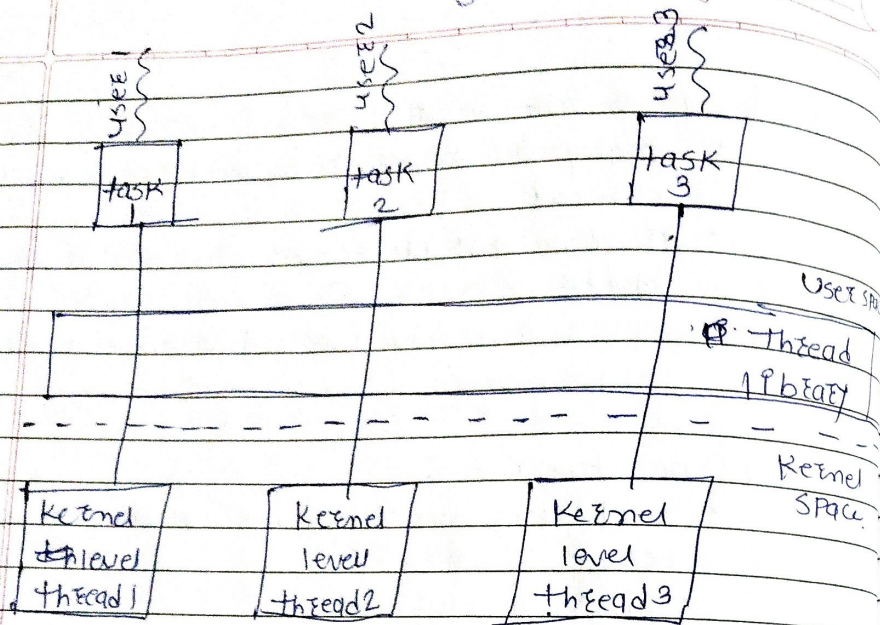
② One-to-one :-

- each user thread maps to a separate kernel thread.
- Best performance.
- one user thread → one kernel thread.

③ many to many :-

- multiple user threads mapped to a small or equal no. of kernel threads.
- It combines benefits of above two models.





* Process control block :-

- It is a data structure that is used by OS to keep track of process info & manage execution.

- It stores everything that OS needs to know about that process.

Process control Block

1) Pointers - It is stack pointer for a process.

2) Process State - stores the respective state of the process

3) P. Number - It is a process ID for that particular process

4) PC - It contains address of next instruction that is to be executed.

5) Register - Current values of CPU registers

6) Memory limit - Store info about memory management unit used by OS.

7) Open files - It includes the list of files opened for a process.

Pointer
Process state
Process number
Process counter
Registers
memory limit
Open files
...

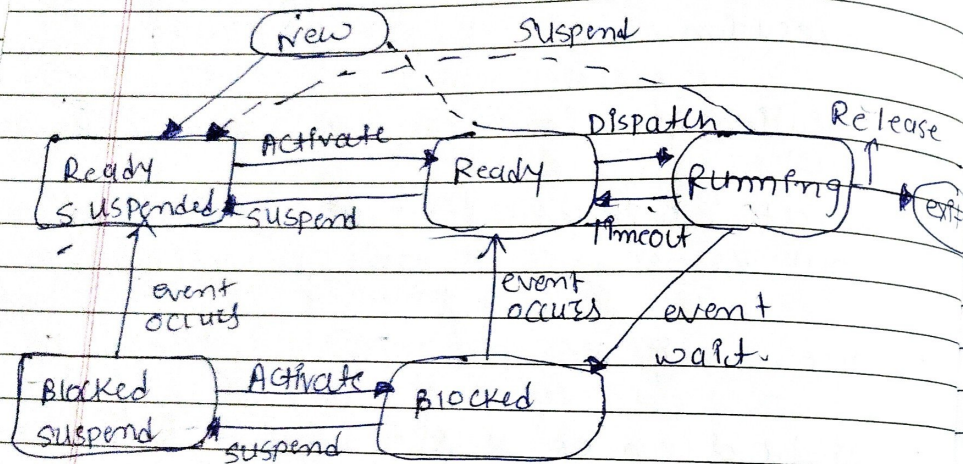
Q6] Compare compilers & interpreters.

Compilers	Interpreters
1) Translates entire source code in machine code at once before execution.	• Translates source code & executes it line by line.
2) Execution speed is faster	• Execution speed is slower
3) Detects & displays all errors.	• Detects & shows one error at a time.
4) Uses more memory	• Uses less memory
5) Produces an executable file	• Does not produce an executable file.
6) Runs after full compilation	• Runs immediately while translating.
7) <u>ex</u> C, C++, Java	7) <u>ex</u> Python, JavaScript

Q7

Q7] Explain seven state ~~not~~ process model with diagram? also explain difference between given 5 state process model.

⇒ The seven state process model is an extension of 5 state model. It consists the following states:



① New :- This state states that the process is created but not ready yet.

② Ready :- This states that process is in memory & ready to execute.

③ Running :- Process is currently running on CPU.

④ blocked (waiting) :- Process is waiting for I/O or event.

⑤ Ready suspended :- Ready but swapped out to disk.

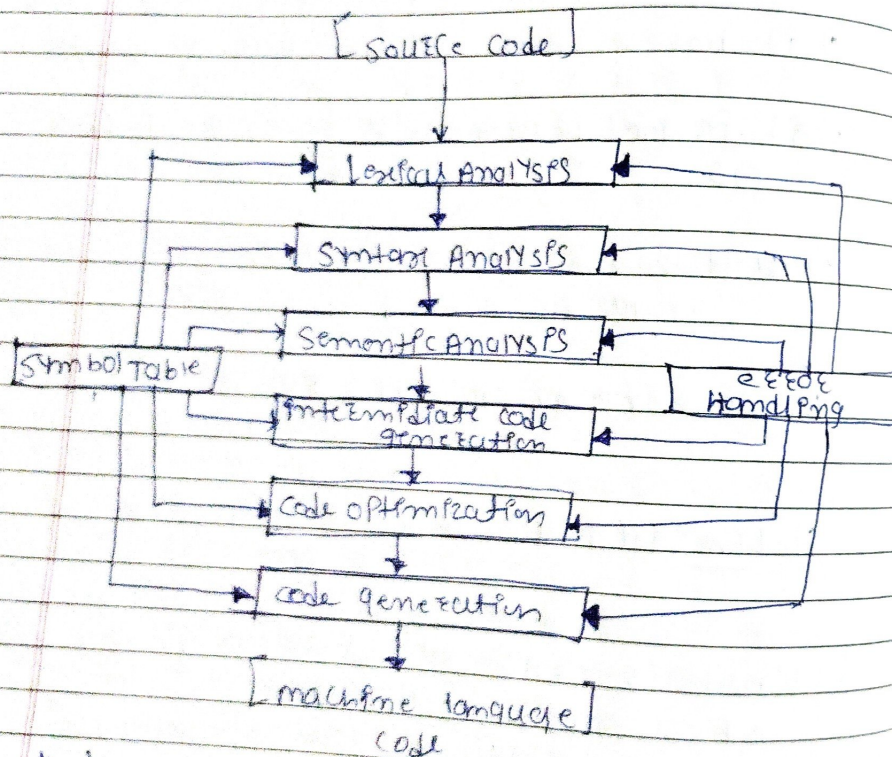
Blocked
⑥ suspended :- Blocked & swapped out to disk.
⑦ Terminated :- process has finished its execution.

5 state model	7 state model
1) It has 5 states	1) It has 7 states
2) It is good for simple OS	2) It is good for advance OS.
3) High memory usage	3) lower memory usage
4) No swapping concept	4) Includes process swapping.
5) Limited performance	5) efficient performance

Q8] What is compiler? explain any two phases of compiler with diagram.

⇒ - compiler is a system software that translates high-level programming language code into low level machine code so computer can execute it.

- * There are 6 phases of compiler
- ① Lexical analysis
 - ② Syntax analysis
 - ③ Semantic analysis
 - ④ Intermediate code generation
 - ⑤ Code optimization
 - ⑥ Code generation



* Lexical Analysis :-

- It is the first phase of compiler
- It's purpose is to read the source code & convert it into tokens.

eg. $a = b + 5;$
 Tokens are $a, =, b, +, ;, 5$

* Syntax Analysis phase :-

- It is the second phase of compiler
- It's purpose is to check whether the sequence of tokens follows the rules of programming language.

eg:

for tokens of $a = b + 5;$
 the syntax analyzer verified if it matched the rule.

statement $\rightarrow id = expression;$
 $expression \rightarrow id + number$

Q9] Define Token, Pattern, & errors of lexical analysis. [mark]

⇒ ① Token :-

- A token is the smallest unit of symbol in a source program that has some meaning to the compiler.
- Tokens are generated by lexical analyzer during lexical analysis phase of compiler.

Examples :-

Keywords $\rightarrow if, while,$

Operators $\rightarrow +, =, -, *, \div$

② Pattern :-

- A pattern is a rule or regular expression that describes the structure of a token.
- It defines how token can be formed from characters in source program.

eg:- number $\rightarrow$ pattern: digit +

③ Lexemes:-

- A lexeme is a actual sequence of characters that matches to the pattern for a token.
- Eg. For source code $sum = a + b;$
sum, a, b are lexemes.

④ Lexical errors:-

- A lexical error occurs when the lexical analyzer encounters that the sequence of lexical characters does not match any valid token pattern.

- Eg. unknown symbol @ that define in language)

★ Q10] What is Lex? Explain working of lex with suitable diagram. [9 marks]

- ⇒
- Lex is a lexical analyzer generator used to create lexical analyzer.
 - It reads patterns or regular expressions written by the programmer & automatically generates C code that can identify tokens.
 - Lex converts the input program into a sequence of tokens for compiler.

* Lex works in three main steps:-

1. Define patterns

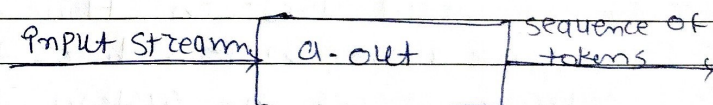
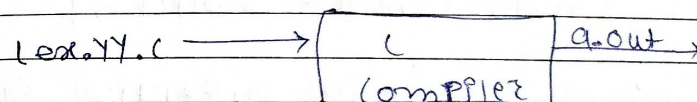
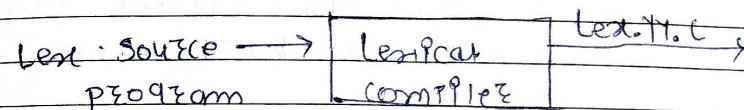
- You write regular expressions to describe tokens.

2. Lex converts patterns to a DFA

- Lex automatically converts your patterns $\rightarrow$ NFA $\rightarrow$ DFA.
- DFA is used to scan input & match tokens efficiently.

3. Lex produces C code (lex.yy.c)

- After Lex generates a C program
- After compiling this code, it becomes a lexical analyzer.



Q11] What is time quantum & its significance in Round Robin scheduling.

⇒ Time Quantum :-

- Time quantum (also called time slice) is fixed amount of CPU time given to a process in round robin scheduling.

If time quantum is = 2 ms then each process will run for 2 ms & then switch to the next process.

* Significance of RRS time quantum:
Round Robin scheduling :-

1) Ensures fairness :-

- every process gets equal quantum, RR improves time for all processes, specially for short ones.

2) Reduced waiting Time :-

- with proper time quantum, RR improves the response time for all processes, specially for short ones.

3) It affects context switching.

4) Improves response for interactive system.

- It is widely used in time-sharing & interactive system bcoz the time quantum keeps the system responsive.

Q12] What is YACC? Explain working of YACC with suitable diagram.

⇒ YACC (Yet another compiler compiler) is a parser generator tool used to create parser for programming.

- It takes grammar rules written in BNF.
- It generates a parser automatically.
- It is commonly used with lex.

Input file.y → YACC → y.tab.c

y.tab.c → C compiler → a.out

Input → a.out → Output

* Working of YACC :-

① Input specification :-

The programmer writes a YACC file which contains tokens, grammar rules etc.

② YACC generates a parser :-

- YACC creates a file (usually y.tab.c)
- This file contains the LALR(1) parser code.
- The parser used shift-reduce & reduce-reduce parsing techniques.

③ Parser Execution :-

- LEX provides tokens to YACC
- YACC matches tokens using grammar rules
- When a rule matches, the action code runs
- If syntax error occurs → YACC calls yyerror().